

# Industrialiser les données de l'entreprise

Alternance : dataplateforme quotidienne et interfaces de collecte

## COLLECTER

MongoDB, ServiceNow et requêtes  
Elasticsearch.  
Orchestration quotidienne avec  
Airflow.

## TRANSFORMER

Chargement dans ClickHouse.  
Transformations dbt.  
Marts pour des usages métier  
réels.

## PARTAGER

Documentation de la plateforme.  
Complément distinct : endpoints  
Tines au service d'un agent cyber.

**Une chaîne réalisée et maintenue de bout en bout pendant l'alternance.**

# Transformer les données internes en usages métier

Des sources opérationnelles aux tables exploitées par les équipes.

## BESOIN

Rendre les données internes disponibles sous une forme adaptée aux cas d'usage des équipes.

## RÉPONSE

Une alimentation quotidienne.  
Des transformations centralisées.  
Des marts destinés à la consommation métier.

## CONTRIBUTION

Conception et réalisation complètes.  
Scalabilité, correction des bugs et demandes d'évolution.  
Documentation.

**Ajouter un exemple réel de mart et la décision métier qu'il permet.**

# Concevoir, exploiter et faire évoluer la chaîne

Périmètre personnel confirmé : l'ensemble de la dataplateforme.

## RÉALISATION

Connexion aux sources.  
Orchestration et chargement.  
Transformations dbt.  
Mise à disposition des marts.

## MAINTENANCE

Résolution des bugs.  
Adaptation à la charge.  
Prise en compte des demandes de fonctionnalités.

## À CHIFFRER

Volumes et croissance.  
Durée du DAG et fraîcheur.  
Calendrier et budget.  
Bénéfices pour les équipes.

**Distinguer la responsabilité exercée des résultats chiffrés encore à ajouter.**

# De trois sources aux marts métier

Schéma fonctionnel de la réalisation décrite.

## SOURCES

MongoDB  
ServiceNow  
Elasticsearch

Données internes hétérogènes.

## ORCHESTRATION

DAG Airflow quotidien  
↓  
Chargement ClickHouse

Modalités par source à détailler.

## TRANSFORMATIONS

Projet dbt  
↓  
Données traitées et améliorées  
↓  
Marts consommés par les équipes.

**La documentation explique le trajet de la donnée et les choix de transformation.**

# Trois sources avec des contraintes différentes

Les connecteurs et les règles de récupération constituent la preuve.

## MONGODB

Collections et champs utiles.  
Filtres et identifiants.  
Mode complet ou incrémental à expliciter.

## SERVICENOW

Source confirmée.  
Préciser le mode d'accès réel :  
API, connecteur ou autre accès à la donnée.

## ELASTICSEARCH

Requêtes d'extraction.  
Périmètre et filtres.  
Pagination, volume et erreurs à documenter selon le code.

**La diversité est réelle ; chaque mécanisme doit être montré tel qu'il est implémenté.**

# Convertir les données ServiceNow avant ingestion

Des chaînes issues du XML provoquaient des erreurs de type dans ClickHouse.

## ENTRÉE

Dans cette intégration, les lignes ServiceNow sont représentées en XML.  
Les valeurs arrivent comme chaînes brutes.

## TRANSFORMATION

Système de validation développé.  
Conversion des chaînes vers les types de données attendus.  
Règles exactes à illustrer.

## CIBLE

Valeurs adaptées au schéma ClickHouse.  
Traitement des données invalides et validation du résultat à documenter.

**Un cas métier réel pour démontrer nettoyage, typage et fiabilisation de la collecte.**

# Expliquer comment un mart est construit

Relier la requête, la granularité et l'usage final.

## ORIGINES

Sélectionner un mart réel.  
Identifier ses sources et les objets intermédiaires nécessaires.

## LOGIQUE

Présenter les jointures, filtres ou calculs réellement utilisés.  
Justifier la clé de rapprochement et la granularité.

## VALIDATION

Montrer un résultat attendu.  
Expliquer le traitement des anomalies observées.  
Joindre les contrôles existants.

**La simple présence de plusieurs sources ne prouve pas leur agrégation.**

# Adapter la collecte au volume et aux schémas

Selon les tables : quelques dizaines de milliers à plusieurs dizaines de millions de lignes.

## VOLUME

Des tailles de tables très différentes.  
Adapter le traitement à chaque source et table.

## FENÊTRES DYNAMIQUES

Plages de récupération quotidienne ajustées selon les tables.  
Règle de calcul réelle à montrer.

## SCHÉMA ÉVOLUTIF

Ajout à la volée de colonnes absentes des schémas ClickHouse.  
Types et contrôles à expliciter.

**La scalabilité est illustrée par des mécanismes concrets ; les gains restent à mesurer.**

# Défendre les requêtes par leur usage

Le projet dbt fournit une base de preuves SQL professionnelle.

## SÉLECTION

Champs utiles et filtres.  
Conditions liées au besoin métier.  
Limiter les données inutiles.

## TRANSFORMATION

Jointures et agrégations réelles.  
Calculs et granularité.  
Résultat attendu du mart.

## OPTIMISATION

Choix ClickHouse et SQL à expliquer.  
Mesures avant / après à joindre.  
Ne pas inventer moteur ou partitionnement.

**Un extrait SQL commenté vaut mieux qu'une liste de technologies.**

# Justifier ClickHouse et le modèle des données

La création, l'import et la documentation relèvent du périmètre réalisé.

## MODÈLE

Décrire les entités, identifiants et relations du cas retenu.  
Joindre les modèles conceptuel et physique.

## STOCKAGE

Présenter les tables ClickHouse et les choix de stockage réellement appliqués.

## IMPORT

Expliquer chargement et actualisation.  
Montrer une exécution et les objets produits.  
Préciser la stratégie de reprise.

**Un graphe de dépendances dbt ne remplace pas automatiquement un modèle Merise.**

# Rendre la plateforme compréhensible et transmissible

La documentation est un axe majeur du travail d'alternance.

## COMPRENDRE

Architecture et sources.  
Définition des données.  
Logique des transformations et usages des marts.

## EXPLOITER

Installation et accès.  
Lancement et diagnostic.  
Procédures de maintenance à montrer lorsqu'elles existent.

## ÉVOLUER

Dépendances et impacts.  
Choix techniques argumentés.  
Mise à jour liée aux changements.

**Montrer une page réellement produite et expliquer à qui elle sert.**

# Documenter les traitements et les droits

Registre du jeu ; ne décrit pas les traitements internes de l'entreprise.

## PÉRIMÈTRE

Comptes, e-mails, mots de passe hachés, sauvegardes et personnages rattachés à un compte.

## PROCÉDURES

Export utilisateur.  
Rectification via administration.  
Suppression du compte et cascades.  
Nettoyage avec prévisualisation.

## À VALIDER

Bases légales et durées proposées.  
Sauvegardes terminées : 365 jours.  
Logs : 30 jours proposés.  
Validation par le responsable.

**Un registre et des commandes constituent des éléments de démarche ; pas une certification de conformité.**

# Tines fournit les données à l'agent cyber

Quatre stories exposent chacune un webhook avec un contrat d'API précis.

## APPELANT

Application Django avec analyste LLM.  
L'agent peut demander des informations sur une entité cyber.

## INTERFACE

Webhook Tines  
↓  
Story correspondant au type d'entité  
↓  
Appels aux consoles EDR nécessaires.

## RÉPONSE

Données des API EDR renvoyées à l'appelant.  
Montrer contrat, accès et erreurs sur un exemple réel.

**Projet distinct : ces endpoints ne sont pas présentés comme une API des marts.**

# Montrer le trajet de la donnée

Préparer les pièces réelles et anonymisées pour la soutenance.

## PLATEFORME

DAG quotidien.  
Une extraction représentative.  
Un modèle dbt et son résultat.  
Une page de documentation.

## MISE À DISPOSITION

Un mart et son usage métier.  
Puis, projet distinct : une story  
Tines et son contrat d'entrée /  
sortie.

## GARANTIES

Montrer une évolution ou un bug  
corrigé.  
Ajouter mesures de charge et  
validation si disponibles.

**Le support décrit tes travaux ; aucune connexion aux systèmes d'entreprise n'a été exécutée ici.**

# Des réalisations professionnelles complémentaires

Un fil conducteur data et une preuve distincte d'interface API.

## C1 / C2

Extraction multi-source  
quotidienne.  
Requêtes Elasticsearch et  
transformations SQL dbt.  
Contraintes de charge à chiffrer.

## C3 / C4

Marts métier et maintenance.  
Modèle et import ClickHouse.  
Nettoyage, Merise et données  
personnelles à expliciter.

## C5

Stories Tines avec contrat précis.  
Caractère REST, accès et tests à  
documenter.  
API du jeu disponible en  
complément.

**Attribuer chaque preuve à son projet et à ta contribution exacte.**

# Revenir à l'origine d'un monstre

Preuve complémentaire existante ; à remplacer par un extrait dbt si disponible.

## SÉLECTION

```
SELECT m.name, s.source,  
s.source_record_id, s.source_url,  
s.collected_at, s.selected,  
s.ingestion_run_id
```

## JOINTURE

```
FROM monsters AS m  
JOIN monster_sources AS s  
ON s.monster_id = m.id
```

## PARAMÈTRE ET TRI

```
WHERE m.id = :monster_id  
ORDER BY s.selected DESC,  
s.source;
```

ID lié comme paramètre SQL.

**Le résultat explique à la fois la provenance et la priorité de fusion.**

# Préparer les preuves d'alternance

Le contenu métier vient des descriptions du candidat, sans accès au code entreprise.

## DATAPLATEFORME

DAG et extrait de run.  
Modèle SQL dbt et résultat.  
Schéma de stockage.  
Documentation et mesures.

## TINES

Une story représentative.  
Contrat anonymisé.  
Requête / réponse.  
Tests et politique d'accès.

## COMPLÉMENT JEU

Sources fichier / scraping.  
Registre et modèles propres au jeu.  
API et tests déjà disponibles.

**Aucune mesure locale du jeu n'est attribuée à la plateforme de l'entreprise.**

# Défendre les décisions sans extrapoler

Les points à préparer avant passage.

## SCALABILITÉ

Quel volume et quelle croissance ?  
Quel goulot ?  
Quel changement et quelle mesure ?

## QUALITÉ

Quel nettoyage réel ?  
Quelle règle de rapprochement ?  
Comment détecter une régression ?

## MISE À DISPOSITION

Qui consomme les marts ?  
Qui appelle les webhooks ?  
Quel contrôle des accès clients ?

**Nommer les faits, les résultats et les limites propres à chaque projet.**